

Streamlit Annotation Service - Implementation Report

Project Overview

Create a Streamlit-based annotation tool for evaluating coherence between memes (images) and associated text assertions. The tool presents annotators with image-text pairs and collects binary judgments on their consistency.

Technical Requirements

Dependencies

streamlit>=1.28.0

Pillow>=10.0.0

pandas>=2.0.0

Directory Structure

```
project_root/
├── app.py          # Main Streamlit application
├── data/
│   ├── images/    # Folder containing meme images
│   └── texts/      # Folder containing text files
├── annotations/    # Output folder for saved annotations
├── annotations.csv # CSV file storing all annotations
└── requirements.txt # Python dependencies
```

Implementation Steps

Step 1: File Management System

Objective: Create a robust system to pair images with their corresponding text files.

Instructions:

- Implement a function that scans the `data/images/` folder for all image files (support .jpg, .jpeg, .png, .gif formats)
- Implement a function that scans the `data/texts/` folder for all text files (.txt format)
- Use a naming convention where each image-text pair shares the same base filename (e.g., `meme_001.jpg` pairs with `meme_001.txt`)

- Create a function that validates pairs exist and returns a list of valid pairs with their full file paths
- Handle edge cases: missing pairs, empty folders, invalid file formats

Step 2: Session State Management

Objective: Track annotation progress and maintain state across Streamlit reruns.

Instructions:

- Initialize `st.session_state.current_index` to track which pair is being annotated (default: 0)
- Initialize `st.session_state.annotations` as an empty list to temporarily store annotations during the session
- Initialize `st.session_state.pairs` to store the list of all image-text pairs loaded at startup
- Ensure session state persists across button clicks and page interactions

Step 3: Data Loading Functions

Objective: Load and prepare image and text data for display.

Instructions:

- Create a function `load_image(image_path)` that:
 - Uses PIL (Pillow) to open and load the image
 - Handles potential errors (corrupted files, missing files)
 - Returns the image object ready for Streamlit display
- Create a function `load_text(text_path)` that:
 - Reads the text file content with UTF-8 encoding
 - Handles potential errors
 - Returns the text as a string

Step 4: User Interface Layout

Objective: Design a clean, centered interface for annotation.

Instructions:

- Use `st.set_page_config(page_title="Meme Annotation Tool", layout="centered")` for optimal viewing
- Create a title: "Meme Coherence Annotation Tool"
- Add a progress indicator showing "Annotating pair X of Y"
- Display the current image using `st.image()` with centered alignment and reasonable max width (e.g., 600px)
- Add spacing with `st.write(" ")` or `st.markdown(" --- ")` for visual separation

- Display the associated text in a clearly marked section (use `st.info()` or a styled markdown block)
- Present the annotation question prominently: "Do you think the assertions are coherent and consistent with what is shown in the meme?"

Step 5: Annotation Response System

Objective: Capture annotator decisions with clear action buttons.

Instructions:

- Create two columns using `st.columns(2)` for side-by-side button placement
- In the left column: "✅ Yes, Coherent" button
- In the right column: "❌ No, Not Coherent" button
- When either button is clicked:
 - Record the annotation with: timestamp, image filename, text filename, response (Yes/No), annotator ID (optional)
 - Append to `st.session_state.annotations`
 - Immediately save to CSV file in `annotations/annotations.csv`
 - Increment `st.session_state.current_index`
 - Use `st.rerun()` to refresh and load the next pair

Step 6: Data Persistence

Objective: Save annotations permanently to avoid data loss.

Instructions:

- Create an `annotations/` folder if it doesn't exist
- After each annotation, append a new row to `annotations/annotations.csv`
- CSV columns should include: `timestamp`, `image_file`, `text_file`, `response`, `session_id`
- Use pandas to handle CSV operations with proper escaping and UTF-8 encoding
- Implement append mode to avoid overwriting previous annotations
- Add a CSV header only if the file doesn't exist yet

Step 7: Navigation and Edge Cases

Objective: Handle completion and navigation smoothly.

Instructions:

- Check if `current_index >= len(pairs)`:
 - Display a completion message: "🎉 All pairs have been annotated!"
 - Show summary statistics: total annotations, response distribution
 - Provide a "Start Over" button to reset `current_index` to 0
- Add a "Skip" button (optional) that increments the index without saving an annotation

- Add a "Previous" button (optional) to review the last annotation (decrement index if > 0)

Step 8: Error Handling and Validation

Objective: Create a robust application that handles unexpected inputs gracefully.

Instructions:

- Wrap file loading in try-except blocks with user-friendly error messages
- Validate that folders exist on startup; show clear instructions if they don't
- Check for empty folders and display helpful guidance
- Handle corrupted or unreadable image/text files by skipping to the next pair
- Log errors to a separate error log file for debugging

Step 9: Additional Features (Optional Enhancements)

Objective: Improve usability and data quality.

Instructions:

- Add an optional comment text area for annotators to provide notes
- Include keyboard shortcuts (e.g., 'Y' for yes, 'N' for no) using Streamlit components
- Add an annotator ID input field at the start to track who provided each annotation
- Display a sidebar with annotation statistics and progress visualization
- Implement a review mode to browse previous annotations
- Add image zoom capability for detailed inspection

Running Instructions

For Cursor AI Implementation:

1. Create the directory structure as specified above
2. Install dependencies: `pip install -r requirements.txt`
3. Place meme images in `data/images/` folder
4. Place corresponding text files in `data/texts/` folder (matching filenames)
5. Run the application: `streamlit run app.py`
6. Access the web interface at `http://localhost:8501`

Data Preparation Notes:

- Ensure image-text pairs have matching filenames (e.g., `meme_001.jpg` and `meme_001.txt`)
- Text files should contain plain text assertions
- Supported image formats: JPG, JPEG, PNG, GIF
- Recommended image size: under 5MB for optimal loading

Expected User Flow

1. Annotator opens the Streamlit app
2. First image-text pair is displayed automatically
3. Annotator reads the text and examines the image
4. Annotator clicks either "Yes, Coherent" or "No, Not Coherent"
5. Annotation is saved, and the next pair loads automatically
6. Process repeats until all pairs are annotated
7. Completion screen shows summary and allows restart

Output Data Format

The `annotations.csv` file will contain:

```
timestamp,image_file,text_file,response,session_id  
2024-12-10 14:30:22,meme_001.jpg,meme_001.txt,Yes,session_abc123  
2024-12-10 14:30:45,meme_002.jpg,meme_002.txt,No,session_abc123
```

This structured report provides Cursor AI with clear, actionable steps to implement the annotation service systematically.